Process & Decision Documentation

Project/Assignment Decisions

Side Quests and A4 (Individual Work)

- Focused on building a polished core game loop instead of adding many extra features.
- This was to make the game would feel more complete and functional.
- This decision improved the overall quality and usability of the game.

Role-Based Process Evidence

This section documents how your work developed over time and provides concrete evidence of your process. It is used to make your thinking, iteration, and decision-making visible, whether or not GenAI was used. You may reuse relevant process evidence across documents where appropriate, as long as it accurately reflects the work for that assignment.

For group assignments (A1 – A3), this section is completed collectively, with clearly attributed role-based entries. For the group, your entries should correspond to elements shown in your process overview visualization such as stages, iterations, or decision points.

For individual assignments (Side Quests and A4), this section is completed individually, using a simplified version of the same structure. For Side Quests, you may omit sections below where no meaningful decision-making occurred. For example, for the first side quest which you need to just upload a blank p5.js to GitHub, you must have to remove a significant amount of the process based evidence as not a lot of design decisions were made.

Examples of acceptable process evidence include (but are not limited to):

- Screenshots of video editing timelines
- Drafts of scripts or research notes
- Annotated sources
- Sketches or diagrams
- Early document outlines
- Prototype screenshots
- Code snippets or commits
- Before-and-after revisions

Project management artifacts (e.g., task boards, timelines, coordination) may be included here when they are relevant to understand how assignment-level decisions were made.

If GenAI was used to support planning, coordination, or role management, this should be noted briefly here as part of the process evidence. Deeper reflection on how GenAI affected teamwork, fairness, and coordination over time belongs in the GenAI Reflections (A4).

Entry Header

Name: Joanne Lang

Role(s): Programmer

Primary responsibility for this work: Implementing player mechanics, sound, gravity, respawn system, and debug menu.

Goal of Work Session

Implement additional game features and debugging tools.

Tools, Resources, or Inputs Used

- ChatGPT (GenAI)
- Lecture notes and class examples

GenAI Documentation

Date Used: March 24, 2026

Tool Disclosure: ChatGPT (GPT-5.3)

Purpose of Use: Debugging, implementing new features, and wording support for README documentation.

Summary of Process (Human + Tool)

- I uploaded my game files and asked how to add a debug screen, moon gravity toggle, and respawn feature.
- ChatGPT suggested code, explained where to place it, and helped update the debug menu text and README documentation.

Decision Points & Trade-offs

- I chose which debug features to include and decided to add respawn because it was useful for testing gameplay.
- Made the README.md description shorter, changed formatting.

Verification & Judgement

- Tested the game in Live Server and GitHub Pages to make sure it worked.
- Compared my work to the class examples.

Scope of GenAI Use

- I decided which features to include, changed the formatting of the provided code to my liking and decided where in each file to place it to make sense.

Limitations, Dead Ends, or Open Questions

- Had to manually ask ChatGPT how to add the text for the respawn function to the debug menu on screen because it wasn't included in the code solution it gave me.
- I did not implement more advanced features like enemies or multiple levels due to time constraints, so the game is focused mainly on core mechanics and debugging features.

Appendix

i'm gonna provide you with some files in my js game, save these files, don't do anything yet. here's index.html: <!doctype html> <html lang="en"> <head> <meta charset="utf-8" /> <title>GBDA302 Week 9 - Example 3: Adding Sound & Music</title> <style> body { margin: 0; overflow: hidden; background: #222; } canvas { display: block; } </style> </head> <body> <script src="https://cdn.jsdelivr.net/npm/p5@1.11.4/lib/p5.min.js"></script> <script src="https://cdn.jsdelivr.net/npm/p5@1.11.4/lib/addons/p5.sound.min.js"></script> <script src="https://p5play.org/v3/planck.min.js"></script> <script src="https://p5play.org/v3/p5play.min.js"></script> <script src="sketch.js"></script> </body> </html>

Got it — I've saved your **index.html** file. I won't do anything with it yet.

Send the next file when you're ready.

don't do anything yet, here's jsconfig.json: { "compilerOptions": { "target": "es6", "checkJs": false, "allowJs": true, }, "include": ["*.js", "**/*.js", "./types/p5/index.d.ts", "./types/p5/global.d.ts"], "exclude": ["node_modules"] }

Saved your **jsconfig.json** file as well.
I won't do anything with the files yet.

Send the next file when ready.

don't do anything yet, here's sketch.js: /* Week 9 — Example 3: Adding Sound & Music Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han Date: Mar. 19, 2026 Controls: A or D (Left / Right Arrow) Horizontal movement W (Up Arrow) Jump Space Bar Attack Tile key: g = groundTile.png (surface ground) d = groundTileDeep.png (deep ground, below surface) = empty (no sprite) */ let player; let playerImg, bgImg; let jumpSfx, musicSfx; let musicStarted = false; let playerAnis = { idle: { row: 0, frames: 4, frameDelay: 10 }, run: { row: 1, frames: 4, frameDelay: 3 }, jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 }, attack: { row: 3, frames: 6, frameDelay: 2 }, }; let ground, groundDeep; let groundImg, groundDeepImg; let attacking = false; // track if the player is attacking let attackFrameCounter = 0; // tracking attack animation // --- TILE MAP --- // an array that uses the tile key to create the level let level = [" ", " ", " ", " ", " ", " ggg ", "gggggggggggggg", // surface ground "ddddddddddddd", // deep ground]; // --- LEVEL CONSTANTS --- // camera view size const VIEWW = 320, VIEWH = 180; // tile width & height const TILE_W = 24, TILE_H = 24; // size of individual animation frames const FRAME_W = 32, FRAME_H = 32; // Y-coordinate of player start (4 tiles above the bottom) const MAP_START_Y = VIEWH - TILE_H * 4; // gravity const GRAVITY = 10; function preload() { // --- IMAGES --- playerImg = loadImage("assets/foxSpriteSheet.png"); bgImg = loadImage("assets/combinedBackground.png"); groundImg = loadImage("assets/groundTile.png"); groundDeepImg = loadImage("assets/groundTileDeep.png"); // --- SOUND --- if (typeof loadSound === "function") { jumpSfx = loadSound("assets/sfx/jump.wav"); musicSfx = loadSound("assets/sfx/music.wav"); } } function setup() { // pixelated rendering with autoscaling new Canvas(VIEWW, VIEWH, "pixelated"); // needed to correct an visual artifacts from attempted antialiasing allSprites.pixelPerfect = true; world.gravity.y = GRAVITY; // Try to start background music immediately. if (musicSfx) musicSfx.setLoop(true); startMusicIfNeeded(); // --- TILE GROUPS --- ground = new Group(); ground.physics = "static"; ground.img = groundImg; ground.tile = "g"; groundDeep = new Group(); groundDeep.physics = "static"; groundDeep.img = groundDeepImg; groundDeep.tile = "d"; // a Tiles object creates a level based on the level map array (defined at the beginning) new Tiles(level, 0, 0, TILE_W, TILE_H); // --- PLAYER --- player = new Sprite(FRAME_W, MAP_START_Y, FRAME_W, FRAME_H); // create the player player.spriteSheet = playerImg; // use the sprite sheet player.rotationLock = true; // turn off rotations (player shouldn't rotate) // player animation parameters player.anis.w = FRAME_W; player.anis.h = FRAME_H; player.anis.offset.y = -4; // offset the collision box up player.addAnis(playerAnis); // add the player animations defined earlier player.ani = "idle"; // default to the idle animation player.w = 18; // set the width of the collsion box player.h = 20; // set the height of the collsion box player.friction = 0; // set the friciton to 0 so we don't stick to walls player.bounciness = 0; // set the bounciness to 0 so the player doesn't

bounce // --- GROUND SENSOR --- for use when detecting if the player is standing on the ground sensor = new Sprite(); sensor.x = player.x; sensor.y = player.y + player.h / 2; sensor.w = player.w; sensor.h = 2; sensor.mass = 0.01; sensor.removeColliders(); sensor.visible = false; let sensorJoint = new GlueJoint(player, sensor); sensorJoint.visible = false; } function startMusicIfNeeded() { if (musicStarted || !musicSfx) return; const startLoop = () => { if (!musicSfx.isPlaying()) musicSfx.play(); musicStarted = musicSfx.isPlaying(); }; // Some browsers require a user gesture before audio can start. const maybePromise = userStartAudio(); if (maybePromise && typeof maybePromise.then === "function") { maybePromise.then(startLoop).catch(() => {}); } else { startLoop(); } } function keyPressed() { startMusicIfNeeded(); } function mousePressed() { startMusicIfNeeded(); } function touchStarted() { startMusicIfNeeded(); return false; } function draw() { // --- BACKGROUND --- camera.off(); imageMode(CORNER); image(bgImg, 0, 0, bgImg.width, bgImg.height); camera.on(); // --- PLAYER CONTROLS --- // first check to see if the player is on the ground let grounded = sensor.overlapping(ground); // -- ATTACK INPUT -- if (grounded && !attacking && kb.presses("space")) { attacking = true; attackFrameCounter = 0; player.vel.x = 0; player.ani.frame = 0; player.ani = "attack"; player.ani.play(); // plays once to end } // -- JUMP -- if (grounded && kb.presses("up")) { player.vel.y = -4; if (jumpSfx) jumpSfx.play(); } // --- STATE MACHINE --- if (attacking) { attackFrameCounter++; // Attack lasts ~6 frames * frameDelay 2 = 12 cycles (adjust if needed) if (attackFrameCounter > 12) { attacking = false; attackFrameCounter = 0; } } else if (!grounded) { player.ani = "jump"; player.ani.frame = player.vel.y < 0 ? 0 : 1; } else { player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle"; } // --- MOVEMENT --- if (!attacking) { player.vel.x = 0; if (kb.pressing("left")) { player.vel.x = -1.5; player.mirror.x = true; } else if (kb.pressing("right")) { player.vel.x = 1.5; player.mirror.x = false; } } // --- KEEP IN VIEW --- player.pos.x = constrain(player.pos.x, FRAME_W / 2, VIEWW - FRAME_W / 2); }

Saved your **sketch.js** file as well. I won't do anything with it yet.

Send any other files when you're ready (assets, additional JS files, etc.).

don't do anything yet, here's style.css: html, body { margin: 0; padding: 0; } canvas { display: block; }

Saved your **style.css** file. I won't do anything yet.

If there are more files (assets list, other JS files, README, etc.), send them over.

there's also assets folder, libraries folder with p5.js files and a types / p5 folder with global.d.ts and index.d.ts. what code would i need to add to which files to add a debug screen to this game that allows you to toggle moon gravity on/off?

Here's how to add a **debug screen with a toggle for Moon gravity** to your game.

We'll:

- Add a debug overlay (text on screen)
- Add a key to toggle Moon gravity on/off
- Switch between Earth gravity and Moon gravity

---

1. In `sketch.js` — Add variables at the top

Add these near your other constants:

```
// --- DEBUG / GRAVITY ---
let debugMode = true;
let moonGravity = false;

const EARTH_GRAVITY = 10;
const MOON_GRAVITY = 1.6;
```

Then **replace** this line in `setup()`:

```
world.gravity.y = GRAVITY;
```

with:

```
world.gravity.y = EARTH_GRAVITY;
```

---

2. Add a key toggle (in `keyPressed()` function)

Modify your `keyPressed()` function:

```
function keyPressed() {
  startMusicIfNeeded();

  // Toggle moon gravity with G key
  if (key === "g" || key === "G") {
    moonGravity = !moonGravity;
    world.gravity.y = moonGravity ? MOON_GRAVITY : EARTH_GRAVITY;
  }

  // Toggle debug screen with `
  if (key === "`") {
    debugMode = !debugMode;
  }
}
```

---

3. Add the Debug Screen (in `draw()`)

Add this at the **bottom of your draw() function**:

```
// --- DEBUG SCREEN ---
if (debugMode) {
  camera.off();
  fill(0, 150);
  rect(5, 5, 140, 60);

  fill(255);
  textSize(10);
  text("DEBUG MENU", 10, 20);
  text("Press G: Moon Gravity", 10, 35);
  text("Gravity: " + (moonGravity ? "MOON" : "EARTH"), 10, 50);
  camera.on();
}
```

---

4. Controls Summary

Key	Action
G	Toggle Moon Gravity
`	Toggle Debug Menu
A/D	Move
W	Jump
Space	Attack

---

5. What This Changes Physically

- Earth gravity = **10** (normal jump, fast fall)
- Moon gravity = **1.6** (floaty jumps, slow fall)
- Your jump (`player.vel.y = -4`) will feel much higher on Moon gravity without changing jump code — which is exactly how real gravity works.

---

6. Optional (Make Moon Jump Feel Better)

If Moon jumps are too floaty, change jump when moon is on:

```
if (grounded && kb.presses("up")) {
  player.vel.y = moonGravity ? -2.5 : -4;
  if (jumpSfx) jumpSfx.play();
}
```

If you want, next step we can add:

- Hitboxes visible in debug
- FPS counter
- Player coordinates
- Collision boxes
- God mode

Those are typical things included in a debug menu in games.

what's another potential feature you might need in developing a game

A really common and useful feature in game development (especially for a course project like yours) is a **level reset / respawn system**.

Right now in your game, if the player falls off the map or gets stuck, the game just continues. In most games, you need a way to reset the player.

---

1. Respawn / Reset System

What it does:

- If player falls off the map → respawn at start
- Press a key (like R) → restart level
- Useful for testing and debugging

Example code (add to `draw()`):

```
// Respawn if player falls off map
if (player.y > VIEWH + 50) {
  respawnPlayer();
}
```

Then create a respawn function:

```
function respawnPlayer() {
  player.pos.x = FRAME_W;
  player.pos.y = MAP_START_Y;
  player.vel.x = 0;
```

```
  player.vel.y = 0;
}
```

And add reset key:

```
function keyPressed() {
  startMusicIfNeeded();

  if (key === "r" || key === "R") {
    respawnPlayer();
  }
}
```

---

2. Other Useful Debug / Dev Features

Here are features developers commonly add while making games:

Feature	Why it's useful
Debug menu	Toggle settings like gravity
Respawn/reset	Fix when stuck
Show hitboxes	Fix collision bugs
FPS counter	Performance testing
Player coordinates	Level design
God mode	Testing levels
Level select	Quickly test levels
Sound on/off	Audio testing
Pause game	Debugging
Slow motion	Test physics

---

3. Easy FPS Counter (good debug feature)

Add to `draw()`:

```
if (debugMode) {
  camera.off();
  fill(255);
  textSize(10);
  text("FPS: " + Math.round(frameRate()), 10, 70);
  camera.on();
}
```

---

4. If This Is For GBDA302

Your prof will love if your game has:

- Debug menu
- Sound toggle
- Restart button
- Instructions screen
- Win/lose condition
- Multiple levels
- Simple UI

Those are the kinds of things that make a student game feel like a **complete game**, not just a demo.

---

5. If You Want One "Impressive but Easy" Feature

Add a **double jump**.

Example:

```
let jumps = 0;
let maxJumps = 2;

if (grounded) {
  jumps = 0;
}

if (kb.presses("up") && jumps < maxJumps) {
  player.vel.y = -4;
  jumps++;
}
```

This adds a whole new mechanic and makes the game feel much more polished.

If you want, I can help you plan a small **debug menu with multiple options** (gravity, FPS, hitboxes, respawn, etc.), which would be a really good feature for your assignment.

i implemented the respawn feature, how do i add the text for it to the debug menu on screen

Nice — that's exactly the kind of feature that belongs in a debug menu.

You already have a debug box drawing text like:

```
DEBUG MENU
Press G: Moon Gravity
Gravity: EARTH/MOON
```

Now you just need to **add the respawn text** to that same debug UI.

Step 1 — Find your debug screen code in `draw()`

It probably looks like this:

```
// --- DEBUG SCREEN ---
if (debugMode) {
  camera.off();
  fill(0, 150);
  rect(5, 5, 140, 60);

  fill(255);
  textSize(10);
  text("DEBUG MENU", 10, 20);
  text("Press G: Moon Gravity", 10, 35);
  text("Gravity: " + (moonGravity ? "MOON" : "EARTH"), 10, 50);
  camera.on();
}
```

Step 2 — Make the debug box taller

Change the rectangle height so more text fits:

```
rect(5, 5, 160, 85);
```

Step 3 — Add respawn text

Add a new line for respawn:

```
// --- DEBUG SCREEN ---
if (debugMode) {
  camera.off();
  fill(0, 150);
  rect(5, 5, 160, 85);

  fill(255);
  textSize(10);
  text("DEBUG MENU", 10, 20);
  text("G: Toggle Moon Gravity", 10, 35);
  text("R: Respawn Player", 10, 50);
  text("`: Toggle Debug Menu", 10, 65);
  text("Gravity: " + (moonGravity ? "MOON" : "EARTH"), 10, 80);
  camera.on();
}
```

Step 4 — (Optional but good) Show player position

This is **very useful** for level design and debugging:

Add this line at the bottom of the debug menu:

```
text("X: " + Math.round(player.x) + " Y: " + Math.round(player.y), 10, 95);
```

If you add that, increase the box again:

```
rect(5, 5, 170, 105);
```

Final Debug Menu Will Show

```
DEBUG MENU
G: Toggle Moon Gravity
R: Respawn Player
`: Toggle Debug Menu
Gravity: EARTH
X: 120 Y: 98
```

That's a **proper debug overlay** like real games use.

Other small debug things you could add easily

If you want your debug menu to look more "game dev":

Feature	Code
FPS	`text("FPS: " + Math.round(frameRate()), 10, 110);`
Player velocity	`text("Vel Y: " + player.vel.y.toFixed(2), 10, 125);`
Grounded	`text("Grounded: " + grounded, 10, 140);`

These help a LOT when debugging physics and jumping.

If you want, next we can add **hitbox visibility toggle**, which is another super common debug feature.

fill out the following for my readme.md file: ## Description Briefly describe what your project does. Explain what the user will see, how they can interact, and what inspired the design. Example: > This project lets users switch between shapes using keyboard inputs. > It explores how visual feedback can communicate accessibility states. --- ## Interaction Instructions Describe how to use or play your project. - How does the user start? - What controls or keys do they use? - Any important visual or audio feedback to watch for? Example: > Press `S` to switch shapes and `R` to reset the canvas.

Here's something you can put into your **README.md** and adjust if needed:

Description

This project is a 2D platformer game where the player controls a fox character that can run, jump, and attack. The game includes animated sprites, sound effects, background music, and a tile-based level with ground and collision physics. The player can move across the level, jump onto platforms, and perform an attack action while on the ground.

The project was inspired by classic side-scrolling platformer games and focuses on implementing core game mechanics such as player movement, gravity, collision detection, animation states, and audio. A debug menu was also added to help test features like gravity changes and player respawning during development.

Interaction Instructions

When the game starts, the player can immediately control the character using the keyboard. The goal is to move around the level and test movement, jumping, and attacking mechanics.

Controls:

- `A` or `Left Arrow` → Move left
- `D` or `Right Arrow` → Move right
- `W` or `Up Arrow` → Jump
- `Space` → Attack
- `R` → Respawn player
- `G` → Toggle Moon gravity (debug feature)
- `` ` `` → Toggle debug menu on/off

The game includes sound effects for jumping and background music that starts after the player presses a key or clicks the screen. The debug menu displays information such as the current gravity mode and player position, which helps with testing and development.

If you want, I can also help you write the **Process** section or **Assets** section, which are usually required for GBDA302 READMEs.

shorter description

Here's a **shorter version** for your README:

Description

This project is a 2D platformer where the player controls a fox character that can move, jump, and attack. The game uses sprite animations, sound effects, background music, and tile-based platforms with gravity and collision. A debug menu was added to toggle moon gravity and respawn the player for testing. The game was inspired by classic side-scrolling platformer games.

Interaction Instructions

Controls:

- `A` / `Left Arrow` → Move left
- `D` / `Right Arrow` → Move right
- `W` / `Up Arrow` → Jump
- `Space` → Attack
- `R` → Respawn player
- `G` → Toggle Moon gravity (debug)
- `` ` `` → Toggle debug menu

Background music starts after a key press or mouse click. The debug menu displays gravity mode and player position for testing.

also add a short GenAI summary of this conversation: ## GenAI For example: I inputted the .js files and side quest instructions, and ChatGPT gave me explanations of the concepts and .js solutions to try for what I wanted to implement. • Gave it more input – copy pasted in the starting Example 5 files to provide more context after having asked it to explain the side quest instructions; this was to give it more context so it could give a better, more specific response to my initial prompt asking it to explain the instructions • Still didn't fully understand its explanation, so prompted it to further clarify what the difference was between the arrays approach/JSON approach • Tried implementing a given JSON approach, but the blob couldn't physically jump high enough to reach the next platform; prompted it for ways to fix this • Prompted for a way to make gap between platforms big enough for the blob to get stuck but not fall through; the result was technically wrong (gave solution for gap slightly bigger than blob width instead of slightly smaller, but I liked this so I kept it) • When adding "win" platforms, prompted it for exactly where in blobPlayer.js I would need to add the given code snippet • Prompted it to fix the mistake in my code by giving it all my updated files (Live Server screen was just blank white, needed to fix this) • Prompted it to fix the level transition glitch

Here's a **short GenAI summary** tailored to *your* project and what we actually did in this conversation:

GenAI

I used ChatGPT to help implement new features and debug my p5.js platformer game. I provided my project files (index.html, sketch.js, style.css, and configuration files), and ChatGPT suggested code to add a debug menu, toggle moon gravity, and add a respawn feature. It also explained where in the code each feature should be added and how the debug screen should display useful development information like gravity state and player position. I then implemented these features and updated the debug menu to include the new controls.

bullet points

Here's the **GenAI section in bullet points**:

GenAI

- I uploaded my game files (index.html, sketch.js, style.css, etc.) and asked ChatGPT how to add a debug screen to my game.
- ChatGPT suggested code for a debug menu and explained where to place it in the draw() function.
- I asked how to add a moon gravity toggle, and ChatGPT provided code to switch between Earth and Moon gravity using a key press.
- I implemented a respawn feature and then asked ChatGPT how to display the respawn control in the debug menu.
- ChatGPT helped update the debug overlay text to include the new controls and debugging information.
- I used ChatGPT to help write the README description, interaction instructions, and this GenAI summary.

get rid of ""

Here's the **GenAI section without quotation marks**:

GenAI

- I uploaded my game files (index.html, sketch.js, style.css, etc.) and asked ChatGPT how to add a debug screen to my game.
- ChatGPT suggested code for a debug menu and explained where to place it in the draw() function.
- I asked how to add a moon gravity toggle, and ChatGPT provided code to switch between Earth and Moon gravity using a key press.
- I implemented a respawn feature and then asked ChatGPT how to display the respawn control in the debug menu.
- ChatGPT helped update the debug overlay text to include the new controls and debugging information.
- I used ChatGPT to help write the README description, interaction instructions, and this GenAI summary.